

Sunday, September 13, 2026

How It Works

Passive chips: The tag itself contains **no battery**. It consists only of a tiny microchip and a coiled antenna.

Inductive powering: The smartphone's NFC reader emits a small electromagnetic field at 13.56 MHz. When held close, this field powers the tag's antenna through magnetic induction.

Instant payload: Once energized, the chip broadcasts a small string of data (typically a standardized **NDEF** record containing a URL, contact info, or Wi-Fi credentials).

The pop-up: The phone recognizes the payload and shows a notification prompt asking to open Safari/Chrome, trigger an automation, or download a contact.

Common Hardware Examples to Research

Adhesive NFC Stickers / Tags (NTAG213 / NTAG215 / NTAG216): Flexible, coin-sized stickers you can slap onto desks, posters, or equipment. They cost roughly \$0.20 to \$0.50 each in bulk and can be rewritten using free apps like **NFC Tools**.

Smart Business Cards: Plastic, wooden, or metal cards embedded with an NFC chip. Tapping them against a phone displays a digital portfolio or contact card (e.g., Popl, Linq, Dot Card).

On-Metal NFC Tags: Standard NFC tags get their signal scrambled if placed directly onto metal surfaces. On-metal variants include a built-in ferrite shielding layer to isolate the antenna.

Hardened Epoxy Discs / Key fobs: Waterproof, rugged discs designed for outdoor use, tap-to-pay fobs, or gym equipment.

Apple App Clip / Google Instant App Tags: Specially formatted NFC tags used in retail and transit that instantly launch lightweight, zero-install mini-apps (for parking meters, bike rentals, or ordering food).

Well, I do agree that the studio's approach is that it does make sense and it is more clear and how to sort of alleviate that sort of curation. It seems like a very grandiose sort of line of

thinking, because in reality, I would start using it for myself and to that extent, people who may also want this. And I guess to that, to be more, to give way to that sort of line of thinking where it's more tied into the reality of what I can do, a way that I was thinking that would, I guess for me, benefit me more and be more intuitive, would be like an ability to use an image, image recognition software to gain, gain, specifically gain context on the painting. This is because in my opinion, beyond me, but kind of mean sometimes, I would thrift thing, thrift paintings and get my, get my furni- no, not furniture, my art through thrifting. And I think people may not know what it is, but it just looks cool and they want to- this could be an opportunity for them to know and understand what that is. So, in this case, it would be using image recognition to first let my software gain that information and then give the Wikipedia of that, if there is a one available, or give context on it, one that isn't sort of AI generated, or generative, I guess. So, a large language model didn't generate that. Instead, there's a process and a software that would get information, mainly, I guess say primarily Wikipedia or other sources, to give you more, to give the user more context on the painting. And then on top of that, or aside that, once they get more of like what it is, then they'd be able to write that down, write the feelings that they have, or what they learned, or, or a combination of the two, you know. The fluidity in, in that would, I guess, would help, what, what, what my app would help that sort of fluidity in, in the thoughts that they want to have in the painting that they want to say, would be some questions in reference to the information that was gained from the image recognition software to give those questions out to sort of give a starting point for the user to start writing. But they can also have the ability to delete that text and that questions and just leave out what they want. Again, this is me thinking over like a note base, but I also want it to be more visually appealing, so that's like still undecided, but yeah.

This is a highly practical and grounded pivot. Scaling back from a grandiose gallery vision to a tool that directly solves your personal use case—identifying thrifted art and journaling about it—is exactly how successful software is born.

Using image recognition to anchor your journaling in historical fact, rather than AI-generated fluff, is a fantastic constraint. It ensures you are interacting with the actual history of the piece before adding your own personal layer.

Here is how you can architect this exact flow, moving from a camera snap to a fluid journaling space.

The Technical Pipeline

1. Image Capture (The Frontend)

When you tap the NFC tag behind the thrifted painting, your web app opens. If the canvas is blank (no journal entry exists yet), the UI immediately prompts you to snap a photo using the HTML5 `<input type="file" accept="image/*" capture="environment">` attribute, which opens the phone's native camera.

2. Image Recognition (The Backend)

You need a reverse-image search that returns structured data, not just a list of Google Images.

The Tool: Google Cloud Vision API (specifically its `WEB_DETECTION` feature).

How it works: You send the image to the API. Instead of just guessing what the image is, it searches the web and returns recognized **Web Entities**. If it finds a match, it returns the exact entity ID, the name of the painting, the artist, and often a direct link to the relevant Wikipedia page.

3. Factual Context Retrieval (The Truth)

Once Google Cloud Vision identifies the painting (e.g., "The Gleaners by Jean-François Millet"), you ping a factual database to get the hard history.

The Tools: The **Wikipedia REST API** or the **Artsy API** (an open database of artworks).

The Result: You pull in the exact year, the art movement, the medium, and a brief factual summary. No large language models (LLMs) are used to generate this history, ensuring it remains archival and accurate.

4. Prompt Generation (The Synthesis)

This is where you *can* use an LLM (like OpenAI or Gemini), but strictly bound by the facts you just retrieved. You send the LLM the Wikipedia summary and give it a strict prompt: *"Based ONLY on these facts, generate three introspective questions to help the user journal about this painting."*

Example Output: "This piece is from the 19th-century Realism movement, focusing on rural working-class life. What about the muted colors drew you to it in the thrift store?"

5. The Journaling UI (The Canvas)

You want a visually appealing, fluid interface similar to Obsidian or Notion, where text feels impermanent until you decide to keep it.

The Tool: TipTap or **Editor.js**. These are headless, block-based rich text editors for React.

The Flow: The factual context (Wikipedia snippet) sits neatly at the top. The LLM-generated

questions appear below as lightweight, grayed-out "blocks" in the editor. Because it is a block editor, you can seamlessly type your answers directly underneath them, or just hit Backspace to instantly delete the prompt block if you'd rather write a freeform stream of consciousness.

By building it this way, the app acts as a researcher first and a journaling canvas second. It bridges the gap between the unknown history of a thrifted object and your personal attachment to it, without ever letting an AI overwrite the human element.

I wonder how these two ways of using this idea could be integrated together from a wider perspective. In this case, if it gets people to start using it and grabbing attention on the idea, how to pivot from this to the studio version, which ultimately seems the most interesting? I don't know in what sense, because I can look at it in many tech senses of the application and the differences between the application of the user one and the studio one. It's the same infrastructure, but when it comes to the concept, migrating that, I wonder how that looks like and see what you have to say about that and how that transition could happen.

The transition from a personal consumer app to a specialized creative studio isn't about changing your codebase—it is about changing your business model. You are moving from building a "tool" to selling an "experience."

This is a classic "bottom-up" strategy. You use the consumer version to train people on the behavior, and then you use the studio version to monetize that behavior at scale with organizations.

Phase 1: The Consumer App as a Cultural Trojan Horse

Right now, tapping a painting to read its history isn't a natural reflex for most people. Your personal journaling app changes that. By letting everyday users thrift art, identify it, and journal about it, you are normalizing the physical action of tapping a canvas. You are building a community of early adopters who view physical art as a gateway to digital depth.

Phase 2: The Indie Artist Bridge

The natural pivot happens when independent artists see what you are doing with thrifted art and want it for their *original* work. Instead of the tag containing a Wikipedia summary, the

artist writes their own manifesto, adds behind-the-scenes sketches, and sticks the NFC tag to the back of their painting before selling it. Your app transitions from a private journal to a tool for artist authentication and storytelling.

Phase 3: The Studio Model

Once indie artists and local collectives are using your infrastructure, larger organizations will take notice. This is where you launch the creative studio.

When you approach a gallery, brand, or fashion label, you aren't selling them software subscriptions or blank NFC tags. You are selling **Exhibition Design as a Service**.

The Pitch: "We build invisible, interactive layers for physical spaces."

The Deliverable: The gallery hires your studio to map out the exhibition. You procure the right anti-metal NFC tags, integrate them seamlessly into the frames or sculptures, and design a bespoke, read-only version of your web app that matches the gallery's exact branding.

The Difference: The consumer app is *read-write* (fluid, editable, personal). The studio product is *read-only* (highly curated, polished, locked down).

This allows you to remain highly selective. As a studio, you can reject corporate clients who just want to use your tags for cheap advertising, ensuring the technology is only used for the intentional, lo-fi storytelling you value.

Sure, yeah. I guess I never really saw it as a business model. I think it is a business model decision rather than that, and I'm not totally tapped in into that space. I'm more in the tech area, so it would be interesting to see kind of definitionally and in I guess historically, in a way, contextually, I think that's a better word, to know what these models are. I know you mentioned bottom-up, but I don't know any information on that and that sort of thought experiment on how that studio could be, because again, my ultimate goal is just to do it for myself and then go from there, but it is nice thinking about these things.

The jump from "building a cool tool for myself" to running a creative technology studio is actually a very well-documented path in the tech industry. It usually relies on a concept called

Product-Led Growth (PLG) and a **Bottom-Up** adoption model.

Since you are coming from a tech background, let's break down the definitions of these models, how they work historically, and how your NFC project fits into them.

1. The "Bottom-Up" Model (Product-Led Growth)

Historically, enterprise tech was sold **Top-Down**: a salesperson pitched a CEO, a massive contract was signed, and employees were forced to use clunky software (think early Oracle or IBM).

Bottom-Up (PLG) flips this. You build a product so frictionless and useful that a single person adopts it to solve a personal problem.

The Progression: An individual designer uses **Figma** for a personal project. They bring it to their small team. The team loves it. Eventually, the company's IT department realizes 50 people are using Figma and pays for an Enterprise license. (Slack and Notion grew exactly this way).

Your Project: You use the NFC app for your own room. A local indie artist sees it and uses it for their next popup show. A mid-sized gallery attends the popup, loves the interactive element, and reaches out. The product sells itself through grassroots exposure rather than a sales pitch.

2. The "Scratch Your Own Itch" Origin

In the tech and design world, some of the best companies started by solving a hyper-specific, personal problem:

37signals / Basecamp: They started as a boutique web design agency. They built a simple project management tool just to keep track of their own client work. Eventually, clients asked to buy the tool, and it became a billion-dollar product.

Teenage Engineering: They started by building wildly niche, hyper-aesthetic synthesizers for audio nerds. Their distinct design language became so coveted that major hardware companies (like Nothing phones and the Panic Playdate console) now hire them as a creative design studio.

3. The Hybrid Model: Software + Service

If your NFC journaling app catches on, you don't necessarily want to become a standard SaaS (Software as a Service) company selling \$5/month subscriptions. Instead, you migrate into a **Hybrid Creative Technology Studio**.

In this model, you maintain a proprietary tech stack, but you sell your expertise and curation

on top of it.

How the Studio Model Operates:

The Tech (SaaS): You have a robust backend (Google Cloud Vision API, Next.js routing, database) that you built for yourself.

The Service (Agency): When a gallery approaches you, you don't just hand them a login to your app. You act as an "Exhibition Designer." You consult on where to place the NFC tags, you design a custom, white-labeled UI that matches the gallery's branding, and you handle the physical installation of the anti-metal chips.

Why this is a strong business model: Software naturally drives its price down over time toward zero. However, high-end galleries and brands have large budgets for *experiences* and *installations*. By packaging your code as a bespoke studio service, you protect the intentionality of the project (you decide who gets to use it) while commanding premium pricing for your time and artistic direction.

You can absolutely keep building this just for your bedroom—but if it ever organically slips out into the world, the hybrid studio model is how you scale it without losing its soul.

I want to try that out for myself and I want to program something. And I'm wondering how the program would work because you said that you can recode it. So I kind of want it to be more, I won't say the use case, but I would want it to be adhesive and hidden. But what I'm really asking is how that coding, how that coding, how that coding, how recoding it, and what are the sort of stack, like tech stack that would be used?

"Programming" an NFC tag does not mean running code on the tag itself—the microchip has no CPU or OS. Instead, you write an **NDEF (NFC Data Exchange Format) record** into the chip's EEPROM memory. When any smartphone taps it, the phone reads that record and executes an action natively (such as launching a URL in the browser).

Hardware to Get

Chip Type: **NTAG213** (144 bytes, best for URLs) or **NTAG215** (504 bytes, ideal for longer URLs, payload data, or vCards).

Form Factor: Adhesive paper or PET stickers (usually 25mm diameter).

Surface Caveat: If hiding the tag behind or inside anything metal (aluminum, steel, copper),

standard tags will fail because metal shorts out the magnetic field. You must use **anti-metal / on-metal NFC stickers**, which include a ferrite backing layer.

How Writing / "Coding" Works

You have two paths: using ready-made tools or writing custom code to handle encoding.

1. The Zero-Code Path (Fastest Testing)

Download **NFC Tools** (free on iOS & Android).

Navigate to **Write** → **Add a record** → **URL/URI**.

Type the destination endpoint (e.g., [https://yourdomain.com/secret-endpoint] (https://yourdomain.com/secret-endpoint)).

Tap **Write** and touch the top of an iPhone or center-back of an Android to the sticker. The payload is permanently stored on the chip unless rewritten or locked.

2. Writing Custom Code to Program Tags

If you want to build an internal dashboard or app that encodes tags programmatically:

Web NFC API (Browser-based, Android Chrome only):

iOS Safari does not support writing via the Web NFC API. However, on Chrome for Android, you can write tags straight from a web frontend:

Yeah, no, I think that's a good point. And I think as a more personal, like more realistic perception of this is building the infrastructure, I guess a little bit down the road, but feeling the infrastructure made for, I guess, to be prepped for that. I guess use of mine software in my infrastructure, which would be to bring it onto another company, organization, or team, and either I guess have the ability to either me develop the custom UI that's like white labeled and branded for them, or if they already have a design team and have their assets, their visual, whatever, which in my thinking most cases it would be, my software, my infrastructure is already built for somebody to come in and put put the put that code, put that UI there. Or if that's not the case, then I guess even then I would I would be the one developing it, but the option for it to be like more of a integrated way of already doing the same process that you are, just adding this sort of tag seems like a better to me, and I'm not really sure if this is like a business model decision, but to so yeah, just to not to have that option and that ability, so when in this in theory when the studio happens, my infrastructure that I would be building for myself today, or today in quotations, would be sort of built for that and anticipated. Guess for

this is kind of just for me, but it is just feels like and this is full context and this is me sort of gathering those requirements. I guess now taking that class makes me think that it's not necessarily requirements yet, maybe more the specifications. I still got to learn the differences of it, but this one feels like a specifications. It's like it's pretty tied to the future of the infrastructure and the use of it, where I guess as a prototype type of thing would be only for me, but eventually it'd be for that purpose.

You are spot on with your realization about the difference between requirements and specifications. In software engineering, a **requirement** is the *what* (the problem you need to solve), and a **specification** is the *how* (the technical blueprint for solving it).

The Requirement: "The system must allow third-party galleries to use our NFC tags to display their own branded experience without seeing other galleries' data."

The Specification: "The system will be built using a **Headless, API-first, Multi-tenant architecture**, separating the frontend UI from the backend database."

What you are describing is the gold standard for modern SaaS (Software as a Service) platforms. By designing the infrastructure this way from day one, you ensure that transitioning from your bedroom prototype to a gallery studio model requires zero rewriting of your core code.

Here is the architectural specification for how you build this today so it is ready for tomorrow.

The Architecture: Headless & Multi-Tenant

1. "Headless" (API-First)

"Headless" means the backend (the database, the Google Cloud Vision logic, the storage) is completely decoupled from the "head" (the frontend UI).

Your backend doesn't render HTML or CSS. It only sends and receives raw data via an API (JSON).

Because it is headless, a gallery can use your API to feed data into their existing website, or you can build a custom Next.js frontend for them that talks to your API.

2. "Multi-Tenant"

"Multi-tenancy" means one single instance of your software runs on your server, but it serves

multiple different organizations (tenants) as if they each had their own private software.

Your bedroom is Tenant 01.

A local indie gallery is Tenant 02.

A fashion brand is Tenant 03.

They all share the same database, but they can never see each other's data.

How to Prep Your Prototype Today

To ensure your infrastructure is ready for the studio model, you need to implement three specific structural rules in your code right now, even while you are the only user.

1. Every Database Table Needs a `tenant_id`

Do not just create an Items table. Create an Organizations (or Tenants) table first. Every single piece of art, journal entry, or NFC tag you log must be tied to a `tenant_id`.

Why? When an API request comes in, the very first thing your backend does is check the `tenant_id`. If Gallery A requests data, the database automatically filters out anything belonging to Gallery B or your personal room.

2. The NFC Payload Must Be Generic

When you write the data to the physical NFC sticker, do not hardcode a specific frontend URL if you can avoid it. Instead, structure the routing URL to include the tenant and item identifiers:

<https://api.yourstudio.com/route?tenant=01&item=8847>

Why? When tapped, your routing server reads the `tenant=01` parameter. It checks the database to see what frontend URL Tenant 01 uses (e.g., your personal journaling app, or the gallery's custom domain) and instantly redirects the phone there, passing the item data along.

3. Abstract the UI Theming (Design Tokens)

If you plan to build the white-labeled UIs yourself, build your React/Tailwind frontend using a theme configuration file (design tokens) rather than hardcoding colors.

Why? When a gallery hires you, you don't build a new app. You duplicate your frontend template, swap the `theme.json` file to match their brand colors and typography, point it to their `tenant_id`, and deploy it in minutes.

By building a headless, multi-tenant backend now, your "personal prototype" is actually a fully scalable enterprise engine. When the time comes to launch the studio, you won't be changing

your infrastructure; you will just be adding a second tenant.